

A Transformation-Based Benchmark for Evaluating the Robustness of LLMs in Generating OCL

Hamza Attarwala
Polytechnique Montreal
Montreal, Canada

hamza-salim.attarwala@etud.polymtl.ca

Omar Alam
Trent University
Peterborough, Canada
omaralam@trentu.ca

Moataz Chouchen
Concordia University
Montreal, Canada

moataz.chouchen@concordia.ca

Mohammad Hamdaqa
Polytechnique Montreal
Montreal, Canada
mhamdaqa@polymtl.ca

Abstract

Large Language Models (LLMs) have shown promising performance in generating Object Constraint Language (OCL) constraints from natural language specifications. However, existing evaluations rely on publicly available UML models, which may overestimate generalization due to potential data leakage and reliance on recurring lexical and structural patterns. This paper introduces a transformation-based benchmarking approach for OCL generation, based on deterministic, semantics-preserving UML model transformations; such as identifier renaming, attribute reification, and association reification, that preserve specification intent while altering model representation. We evaluate a diverse set of closed-source and open-source LLMs under zero-shot, few-shot, and chain-of-thought prompting, measuring syntactic accuracy (well-formed and type-correct OCL) and semantic accuracy (correct interpretation with respect to the UML model). While leading models achieve high performance on non-transformed models (up to 92.10% syntactic accuracy and 74.56% semantic accuracy), performance degrades substantially under transformation, with best semantic accuracy dropping to 55.96% for closed-source models and 42.20% for open-source models. These results demonstrate that current LLMs remain brittle under semantically equivalent but structurally altered models, suggesting a sensitivity to surface-level patterns and highlighting the need for transformation-based benchmarks for evaluating OCL generation.

CCS Concepts

• **Software and its engineering** → *Unified Modeling Language (UML)*; **System modeling languages**.

Keywords

Object Constraint Language (OCL), Unified Modeling Language (UML), Large Language Models (LLMs), Model-Driven Engineering (MDE)

ACM Reference Format:

Hamza Attarwala, Moataz Chouchen, Omar Alam, and Mohammad Hamdaqa. 2026. A Transformation-Based Benchmark for Evaluating the Robustness of LLMs in Generating OCL. In *ACM/IEEE 29th International Conference on Model Driven Engineering Languages and Systems (MODELS 2026)*, October 04–09, 2026, Málaga, Spain. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3822455.3830333>

1 Introduction

Within Model-Driven Engineering (MDE), the Object Constraint Language (OCL) is essential for specifying invariants, preconditions, and postconditions that UML diagrams alone cannot express [9, 31]. Correct OCL constraints ensure model consistency and support downstream activities such as verification and transformation [1, 37], but writing them is challenging due to strict syntax, declarative semantics, and reliance on precise model navigation. Recent advances in Large Language Models (LLMs) offer the potential to automate OCL generation from natural language specifications and UML models [3, 17, 29, 32, 39]. Early results suggest that LLMs have been shown to produce syntactically valid and seemingly correct OCL expressions under standard evaluation settings, especially when guided by careful prompting [4, 14].

Despite these encouraging results, current evaluation practices present significant limitations that obscure the true capabilities of LLMs. Existing benchmarks are typically small, static, and exhibit limited structural diversity, often reusing similar naming conventions and model patterns [32]. This creates favourable conditions for memorization, pattern matching, and potential data leakage [25, 44]. Moreover, evaluation metrics frequently emphasize surface-level correctness, allowing syntactically valid but semantically flawed constraints to pass undetected. As a result, systematic failure modes, such as incorrect navigation paths, misuse of OCL operators, hallucinated model elements, and logical inconsistencies, may remain hidden [3, 26]. This situation reflects a broader limitation of static evaluation settings, which often fail to reveal robustness issues that emerge under controlled structural variation.

To address these limitations, we propose a transformation-based benchmark for OCL generation that systematically evaluates LLM robustness. Our approach applies deterministic, semantics-preserving UML transformations that alter model structure while preserving specification intent, eliminating lexical and structural cues that



This work is licensed under a Creative Commons Attribution 4.0 International License. *MODELS 2026, Málaga, Spain*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2809-9/2026/10

<https://doi.org/10.1145/3822455.3830333>

LLMs may exploit. As a result, models must regenerate OCL constraints by reasoning over UML structure and navigation paths rather than relying on memorization or pattern matching. By coupling UML transformations with consistent OCL rewrites, the benchmark enables controlled assessment of structural reasoning under realistic MDE model evolution scenarios, such as refactoring and diagram restructuring and changes in how domain concepts are encoded within UML class diagrams, rather than adversarial perturbations.

We design three transformations targeting empirically observed failure modes in OCL generation: (i) identifier renaming, which removes lexical cues and tests reliance on naming patterns; (ii) attribute reification, which introduces additional indirection and challenges multi-step navigation reasoning; and (iii) association reification, which alters structural relationships and disrupts common traversal patterns. Each transformation preserves the semantics of the original model and specification while systematically increasing the reasoning requirements for generating correct OCL constraints, ensuring that any performance change reflects sensitivity to structural variation rather than changes in specification meaning.

We evaluate multiple LLMs on both original and transformed benchmarks, showing that performance drops substantially under transformation. While leading models achieve strong results on standard benchmarks, their accuracy degrades significantly when evaluated on semantically equivalent but structurally altered models. This suggests that apparent success in conventional settings appears to be driven, at least in part, by shallow pattern matching rather than robust reasoning over UML models.

This paper makes three main contributions:

- Introduce a transformation-based, fault-revealing benchmark for OCL generation that systematically exposes hidden LLM failure modes and reduces memorization.
- Demonstrate how deterministic, semantics-preserving UML transformations can be used to generate structurally diverse evaluation cases while preserving specification intent.
- Provide empirical analysis showing that LLM performance degrades under transformation, highlighting gaps between surface-level correctness and true model-aware reasoning.

This work is driven by the following research questions:

RQ1: How do LLMs perform on OCL generation from specifications under standard (non-transformed) evaluation settings?

RQ2: How do semantics-preserving UML model transformations affect the ability of LLMs to generate correct OCL expressions?

RQ3: Which types of OCL generation errors are revealed or amplified under transformation-based benchmarks?

RQ4: To what extent do individual and combined UML transformations differentially affect LLM performance in OCL generation?

Overall, our work positions transformation-based benchmarks as a principled approach for evaluating robustness and semantic reasoning in LLM-based OCL generation, highlighting the need for evaluation methodologies that better capture reasoning capabilities before deploying LLMs in real-world MDE workflows.

Replication Package: We provide a replication package that contains the datasets and Python scripts used in this study.¹

¹<https://doi.org/10.5281/zenodo.19270005>

2 Motivation

Evaluations of LLMs for OCL generation commonly rely on static benchmark datasets composed of UML models, natural language specifications, and corresponding gold-standard constraints. While these benchmarks provide an initial indication of model performance, they often do not reveal whether correct outputs result from genuine reasoning over UML models or from memorization of recurring structural patterns and naming conventions [22, 32]. This limitation is particularly problematic in OCL generation, where publicly available datasets are scarce and structurally homogeneous [3].

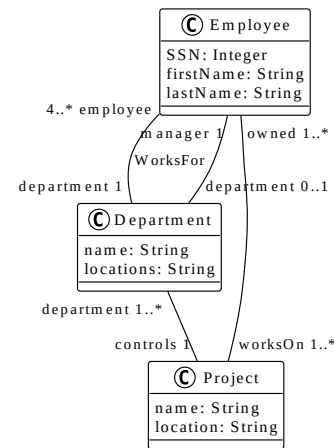


Figure 1: Class diagram for Employment Agency

To illustrate this limitation, consider the simplified UML model shown in Figure 1 from the Employment Agency domain. The model includes a Project class associated with a Department, where each department defines a set of locations. A natural language specification states: *"The location of a project must be in one of the locations of its department."* The correct OCL invariant navigates from Project to Department and uses includes() to ensure that the project location is contained in the department's locations.

```

context Project inv LocationWithinDepartmentLocations:
self.department.locations->includes(self.location)
  
```

In this example, a SOTA LLM (e.g., GPT-5) is able to produce an OCL expression that is both syntactically and semantically correct. On the surface, this suggests that the model correctly understands both the UML structure and the natural language specification. However, this success is difficult to interpret: similar modeling patterns and constraint formulations frequently appear in publicly available examples and tutorials [32], raising the possibility that the model is reproducing familiar structures rather than reasoning about the specific UML model.

Now consider a semantics-preserving transformation of the same UML model (Figure 2). In the transformed model, the association between Employee and Project is reified into an association class, and model elements are systematically renamed. These transformations preserve the intended semantics of the model and do not affect the structure relevant to the specified constraint. As a result, the

correct OCL invariant remains structurally unchanged and only requires adaptation to the renamed elements. However, the following OCL invariant, generated after transformation ², is incorrect.

```
context project_r800 inv ProjectLocationInDepartmentLocations:
self.controls_r781.locations_r810 = self.location_r240
```

Despite semantic equivalence between the original and transformed models, the model fails to produce a valid OCL expression. In particular, the generated constraint exhibits invalid navigation and compares incompatible elements, indicating a failure to correctly interpret the UML structure under transformation.

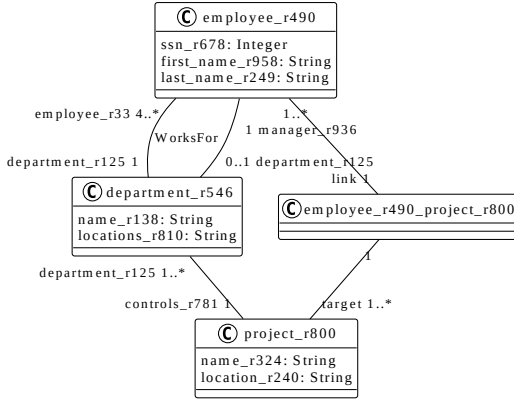


Figure 2: Transformed class diagram for Employment Agency

This example highlights a key limitation of standard evaluation settings: correct performance on a given model does not guarantee robustness to semantically equivalent variations. Empirical evidence further shows that LLMs frequently produce invalid navigation paths, hallucinated model elements, or logically incorrect yet syntactically valid constraints [3], even when performing well on standard benchmarks.

These observations motivate the need for transformation-based benchmarks that can distinguish between memorization and genuine reasoning in OCL generation. By applying deterministic UML transformations that preserve semantics while altering model structure, we construct evaluation cases that require models to reason correctly about navigation and constraints. The next section formalizes these transformations.

3 Benchmark Construction via Deterministic UML Transformations

This section presents a methodology for constructing OCL generation benchmarks using deterministic UML transformations. These transformations preserve semantics while altering lexical and structural cues exploited by LLMs. Semantic preservation is ensured through observational equivalence [38]. Each transformed constraint remains logically equivalent to the original and has identical satisfaction conditions. The selected transformations include identifier renaming, attribute reification, and association reification [6, 28, 38].

²<https://chatgpt.com/share/69c965b1-b870-83e8-a917-43503f19f6d2>

Transformation 1: Identifier renaming. LLMs frequently exploit familiar naming conventions that include meaningful class or attribute names to infer constraint structure without fully reasoning about the underlying model [10]. To eliminate this source of bias, the first transformation systematically renames all UML identifiers while preserving their structural relationships and semantic roles [38].

Given a UML model M , all class names, attributes, and association role names are renamed using a deterministic scheme that combines a normalized snake_case representation of the original identifier with a hash-based suffix derived from the declaring element o . A fixed marker (`_r`) precedes the suffix to explicitly denote the renaming artifact and disambiguate it from numeric components that may already exist in the original identifier. The retention of a normalized lexical form preserves syntactic readability and human interpretability, while the hash-based suffix removes reliance on familiar naming conventions or memorized identifier patterns. Consequently, identical identifiers declared in different contexts receive distinct names while remaining consistently referenced across the UML model and OCL constraints. The renaming is applied globally using a longest-match-first strategy to avoid partial replacements.

As a result, both the transformed UML model M' and the rewritten OCL constraints C' remain semantically equivalent to the original artifacts but no longer contain recognizable lexical cues. This transformation directly targets hallucinated content generation and navigation errors, as LLMs can no longer rely on memorized domain-specific names and must instead reconstruct valid navigation paths from the provided model structure. The complete renaming procedure is formalized in Algorithm 1.

Algorithm 1 Deterministic Identifier Renaming

Require: UML model M , OCL constraint set C , $k > 0$

Ensure: Renamed UML model M' , renamed constraints C'

- 1: Extract classes, attributes, and association roles from M
 - 2: **for all** identifier id owned by element o **do**
 - 3: $s \leftarrow \text{snake_case}(id)$
 - 4: $\text{suffix} \leftarrow (\text{HASH}(o.id) \% k)$
 - 5: $id' \leftarrow s \parallel _r \parallel \text{suffix}$
 - 6: $R[id] \leftarrow id'$
 - 7: **end for**
 - 8: Apply R to all identifiers in M to produce M'
 - 9: Apply R to all constraints in C to produce C'
 - 10: **return** (M' , C')
-

Transformation 2: Attribute-to-Class Reification. Algorithm 2 describes the process of the attribute-to-class reification transformation. While attribute-based navigation in OCL is straightforward, reifying attributes into separate individual classes introduces additional indirection that requires more precise structural reasoning [6]. This transformation is motivated by empirical observations that LLMs often struggle when constraints require multi-step navigation, even when the underlying semantics are unchanged [15]. In particular, attribute multiplicities are preserved by mapping them to corresponding association multiplicities between the original

class and the reified attribute class, ensuring semantic equivalence of the resulting model.

Algorithm 2 Attribute-to-Class Reification

Require: UML model M , OCL constraint set C

Ensure: Transformed UML model M' , rewritten constraints C'

```

1: Extract classes, attributes, and association roles from  $M$ 
2: Identify attribute  $a$  with the highest reference frequency in  $C$ 
3: Let  $cls$  be the class to which attribute  $a$  belongs
4: Remove attribute  $a$  from  $cls$  in  $M$ 
5: Introduce a new class  $A\_Value$  with attribute  $value$ 
6: Add an association between  $cls$  and  $A\_Value$ 
7: for all constraint  $c \in C$  do
8:   Replace occurrences of  $self.a$  with  $self.a.value$ 
9: end for
10: Construct  $M'$  and  $C'$ 
11: return ( $M'$ ,  $C'$ )
  
```

To maximize fault revelation, the transformation selects the attribute a that is referenced most frequently across the gold OCL constraints. This attribute a is then removed from its class and replaced by a newly introduced class that is named `a_value` containing a single attribute representing the original value. A binary association is then added between the original class and the new `a_value` class. All OCL constraints are rewritten to reflect this change by extending direct attribute access into a two-step navigation path.

The naming of the introduced class intentionally preserves a lexical link to the original attribute. This avoids introducing synthetic or semantically opaque identifiers, which could confound evaluation by simultaneously altering naming cues and structural complexity. By retaining a transparent name while changing the navigation structure, the transformation isolates structural reasoning demands and ensures that any observed performance degradation can be attributed to increased indirection rather than loss of interpretability.

Because the transformed model preserves the meaning of the original constraints while increasing structural complexity, LLMs are forced to adapt their constraint generation logic rather than reproducing familiar attribute access patterns. In practice, this transformation frequently exposes navigation errors, incomplete rewrites, and logically incorrect constraints that remain syntactically valid.

Transformation 3: Association Reification. Binary associations are commonly navigated directly in OCL constraints, and LLMs often assume such navigation patterns by default. The third transformation challenges this assumption by reifying a frequently used binary association into an explicit association class, thereby requiring indirect navigation and a more explicit understanding of model structure.

The association selected for reification is chosen based on its usage frequency in the gold OCL constraints. Given a binary association between classes A and B with role names and multiplicities preserved, the original association is replaced by a new association class A_B . Two associations are introduced: one from A to A_B and one from A_B to B , with role names and multiplicities derived from

the original association ends. Navigations in OCL constraints are rewritten accordingly: direct navigation expressions (e.g., $a.b$) are replaced by compositions over the association class (e.g., $a.a_b.b$), with collection-valued navigations rewritten using standard OCL collection operations (e.g., `collect`) to preserve cardinality and selection semantics.

This transformation preserves the semantics of the original constraints while substantially altering their navigational structure. It is particularly effective at revealing errors related to incorrect path construction, hallucinated roles, and invalid assumptions about multiplicities or directionality. Even when LLMs produce syntactically valid expressions, the resulting constraints often fail to capture the intended semantics under this transformed representation. The full reification procedure is specified in Algorithm 3.

Algorithm 3 Association Reification

Require: UML model M , OCL constraint set C

Ensure: Transformed UML model M' , rewritten constraints C'

```

1: Extract associations from  $M$ 
2: Identify association role  $r$  most frequently referenced in  $C$ 
3: Let  $(A, B)$  be the classes connected by  $r$ 
4: Remove association  $r$  between  $A$  and  $B$  from  $M$ 
5: Introduce new association class  $A\_B$ 
6: Add associations  $A \rightarrow A\_B$  and  $A\_B \rightarrow B$ 
7: for each constraint  $c \in C$  do
8:   Replace  $self.r$  with  $self.link.target$  in  $c$ 
9: end for
10: Construct  $M'$  and  $C'$ 
11: return ( $M'$ ,  $C'$ )
  
```

These transformation-driven approaches bridge the gap between error characterization and benchmark design, enabling a principled evaluation of LLM generalization and robustness in OCL generation. The resulting benchmarks serve not only as a diagnostic tool for assessing current models but also as a foundation for future research on improving formal specification generation from the perspective of semantic, complexity, and relation based transformations.

4 Experimental Design and Evaluation Setup

Figure 3 illustrates the UML transformation process used to create the benchmark. Each component for building and evaluating the benchmark is described below.

Dataset and UML Models. Our evaluation uses the PathOCL dataset [4], which pairs UML class models with natural language specifications and gold OCL constraints. We exclude two models lacking gold OCL expressions, leaving the remainder for all experiments. Each UML model is represented as a PlantUML class diagram with corresponding natural language specifications. This dataset provides both syntactic ground truth and semantic intent, enabling systematic validation of generated OCL constraints. While limited in size, PathOCL provides high-quality, manually curated UML models, natural language specifications, and gold OCL constraints. To the best of our knowledge, it is currently the only publicly available benchmark that provides aligned triples of these three artifacts, making it the most suitable dataset for evaluating end-to-end OCL generation. Consequently, our experiments use

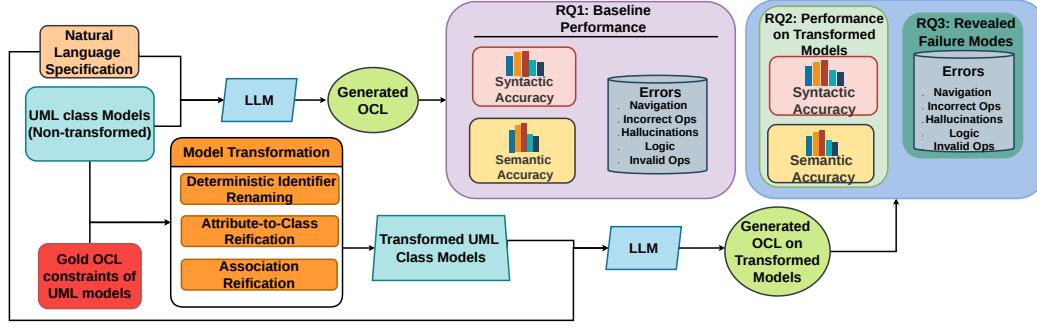


Figure 3: Transformation-Based benchmarking for OCL generation.

PathOCL as a controlled benchmark for studying robustness under semantics-preserving transformations.

Evaluated Language Models and Prompting Strategies. We evaluate a diverse set of contemporary LLMs varying in scale, training objectives, and availability. These include four large closed-source models (Claude Sonnet 4.5, GPT-5, Gemini-2.5-Pro, Grok-4-Fast), two large open-source models (Llama-4, DeepSeek-V3.1), and five smaller open-source models (Microsoft Phi-4 14B, Qwen-3 8B, Mistral-8B-Instruct, Llama-3 8B, CodeLlama 13B). This selection allows comparison across model families, scales, and deployment paradigms under a consistent evaluation protocol, using deterministic decoding with temperature 0, nucleus sampling disabled (top-p = 1.0), and a 512-token limit. Each model is evaluated using three prompting strategies—zero-shot, few-shot, and chain-of-thought—adapted from prior work on LLM-based OCL generation by [3] and [32], with minor modifications for our experimental setting. Prompts include the UML diagram in PlantUML format and the natural language specification, instructing the model to generate OCL constraints using only the UML identifiers. Few-shot prompts include a small illustrative example of invariant, precondition, and postcondition generation. Models are instructed to output only the final OCL expressions to ensure uniform evaluation.

Fine-Tuning of Smaller Language Models. In addition to prompt-based evaluation, we fine-tune the smaller open-source LLMs to assess whether domain-specific supervision improves OCL generation performance. Fine-tuning is conducted using the dataset introduced by [32], which contains 52 synthetically generated meta-models represented as EMF files and PlantUML descriptions, 369 OCL constraints, and 369 corresponding natural language specifications.

This dataset is particularly well aligned with our task, as it provides paired representations of UML structure, natural language intent, and executable OCL constraints. We therefore fine-tune the smaller models using this dataset to examine whether explicit exposure to OCL syntax and modelling patterns improves syntactic correctness and semantic alignment. Larger models are evaluated exclusively in a prompt-based setting, allowing us to contrast in-context learning with parameter adaptation across model sizes.

Evaluation Procedure. The generated OCL constraints are evaluated using a multi-stage validation procedure designed to distinguish syntactic correctness, structural conformance to the UML

model, and semantic alignment with the natural language specification.

In this study, semantic accuracy denotes correctness with respect to both the UML model and the intended specification. A generated constraint is considered semantically correct if and only if it (i) is syntactically well-formed, (ii) conforms to the UML model structure and type system, and (iii) exhibits correct runtime behaviour with respect to the natural language specification.

First, all generated constraints are parsed using the USE [19] tool to assess syntactic well-formedness. Constraints rejected by the parser are classified as syntactically invalid and excluded from further analysis.

Second, syntactically valid constraints are evaluated for structural semantic correctness by executing them in USE against the corresponding UML model. At this stage, constraints are type-checked and validated to ensure that all navigations, associations, and OCL operations are well-defined with respect to the model. Constraints that reference non-existent model elements, violate typing rules, or misuse collection operations are classified as structurally incorrect.

Finally, structurally valid constraints are evaluated for behavioural adequacy. For each constraint, representative object diagrams are constructed in USE to cover both satisfying and violating cases of the intended specification. A constraint is deemed behaviourally correct if it accepts all satisfying instances and rejects all violating instances derived from the natural language specification. Any discrepancy between constraint behaviour and intended semantics is recorded as a semantic error.

Construction and Validation of Test instances. To evaluate semantic correctness, we manually constructed object diagrams in USE for every natural language specification. For each specification, we first identified the context class, referenced UML elements, and constraint type (e.g., attribute value, multiplicity, association navigation, collection property, or cardinality constraint). We then created positive, negative, and boundary-case object instances that exercised the intended behaviour while keeping unrelated parts of the model unchanged. All object instances were validated against the corresponding gold OCL constraint before use. A generated constraint was considered semantically correct only if it exhibited the same behaviour as the gold constraint across the validated test suite.

For transformed UML models, we create equivalent instances by systematically adapting the original ones. When attributes are

reified, their values are represented as separate objects connected through the new associations. When associations are reified, links between objects are replaced by instances of the corresponding association class, while preserving the original relationships. These transformed instances are checked to ensure they still represent the same intended behaviour as before.

Transformation Summary. To quantify the extent to which the transformed UML models differ from their original counterparts, we compute several model-level statistics (Table 1) that capture lexical and structural divergence while remaining independent of any LLM-generated outputs.

Across the dataset, the average token-level edit distance between original and transformed PlantUML models is 0.47, indicating substantial syntactic modification to the UML representations. In addition, the identifier replacement ratio averages 0.97, showing that nearly all class, attribute, and association identifiers are renamed or altered by the applied transformations. This confirms that transformed models exhibit minimal lexical overlap with their original versions.

Beyond renaming, the transformations also introduce structural changes. Across all domains, 26 new classes are introduced through reification and decomposition operations, while 60 existing classes are systematically renamed. These changes preserve the semantic intent of the original specifications but significantly alter the syntactic and structural cues available to language models.

Table 1: Summary Statistics of UML Model Transformations

Metric	Value
Average Token Edit Distance	0.47
Average Identifier Replacement Ratio	0.97
New Classes Introduced	26
Renamed Classes	60

Validation of Semantic Preservation. To ensure that our UML transformations preserve the original meaning of the specifications, we compare the behaviour of the original and transformed models using the corresponding gold OCL constraints. For each transformation, we rewrite the original constraint to match the transformed model and execute it in USE. We then check that both the original and rewritten constraints produce the same results on the constructed object diagrams—that is, they accept the same valid instances and reject the same invalid ones.

This process provides practical evidence that the transformations preserve the intended semantics. However, we note that this validation is based on a limited set of test instances rather than all possible cases. To reduce the risk of errors, we also manually inspect a subset of the transformed models and constraints.

5 Results

5.1 RQ1: How do LLMs perform on OCL generation from specifications under standard (non-transformed) evaluation settings?

RQ1 investigates how effectively contemporary LLMs generate correct OCL constraints from natural language specifications and

Table 2: Large LLMs On Non-Transformed Models

Model	Prompt	Syntactic Accuracy (%)	Semantic Accuracy (%)
Closed-Source Models			
Claude Sonnet 4.5	ZS	86.84	71.92
	FS	85.08	64.03
	CoT	90.35	71.05
GPT-5	ZS	87.71	70.17
	FS	88.59	73.87
	CoT	92.10	74.56
Gemini 2.5 Pro	ZS	90.35	69.29
	FS	88.59	68.42
	CoT	92.10	71.05
Grok-4 Fast	ZS	81.57	62.28
	FS	85.08	66.66
	CoT	85.96	64.03
Open-Source Models			
Llama-4	ZS	85.08	62.28
	FS	83.33	58.77
	CoT	86.84	60.52
DeepSeek V3.1	ZS	37.50	26.27
	FS	85.08	58.77
	CoT	61.81	37.27

ZS: Zero-Shot, FS: Few-Shot, CoT: Chain-of-Thought

Table 3: Error Taxonomy Distribution Across Large LLMs and Prompting Techniques

	Claude 4.5			GPT-5			Llama-4			Gemini 2.5			DeepSeek V3.1			Grok-4 Fast		
	ZS	FS	CoT	ZS	FS	CoT	ZS	FS	CoT	ZS	FS	CoT	ZS	FS	CoT	ZS	FS	CoT
Navigation	8	12	9	11	5	5	20	18	16	11	9	13	6	10	10	8	11	9
Incorrect Ops	8	6	4	6	4	4	4	3	5	7	5	4	5	3	3	10	8	11
Hallucination	2	4	5	5	5	3	1	6	7	4	2	11	9	8	9	8	5	8
Logic	4	7	8	8	10	9	8	5	8	7	8	3	9	8	5	5	9	9
Invalid Ops	9	13	8	5	7	6	8	16	8	6	7	4	64	13	40	7	10	5

ZS: Zero-Shot, FS: Few-Shot, CoT: Chain-of-Thought

UML class models under standard, non-transformed conditions. The results reveal clear performance differences across model scales, prompting strategies, and fine-tuning approach.

Among large-scale models (Table 2), syntactic correctness is generally high across prompting strategies, with GPT-5 and Gemini 2.5 Pro consistently achieving the strongest results. Chain-of-thought prompting marginally improves syntactic accuracy for some models, but its impact on semantic accuracy is less consistent. For instance, GPT-5 achieves its highest semantic accuracy under few-shot prompting, while Gemini 2.5 Pro shows comparable semantic performance across prompting variants. These observations suggest that, for large models, syntactic correctness is largely robust to prompt formulation, whereas semantic alignment remains sensitive to both model size and prompt context.

Despite high syntactic accuracy, semantic errors remain common even for the strongest models. The error taxonomy (Table 3) analysis indicates that navigation and logic errors account for a substantial portion of failures, demonstrating that correct parsing and type checking alone do not guarantee faithful interpretation of the natural language specification. This gap highlights a persistent limitation in translating informal requirements into precise, model-consistent constraints, even when the generated OCL is syntactically valid.

Smaller models, on the other hand, exhibit considerably lower baseline performance, particularly under zero-shot and chain-of-thought prompting (Table 4). Few-shot prompting improves results for some models but remains insufficient to close the gap with

Table 4: Small LLMs On Non-Transformed Models

Model	Prompt	Syntactic Accuracy (%)	Semantic Accuracy (%)
Microsoft Phi-4 (14B)	ZS	35.08	20.17
	FS	42.98	28.07
	CoT	40.35	28.94
Qwen-3 (8B)	ZS	5.26	4.38
	FS	57.01	35.08
	CoT	6.14	5.26
Ministral-8B-Instruct (8B)	ZS	13.15	8.77
	FS	34.21	18.42
	CoT	8.77	7.01
Llama-3 (8B)	ZS	6.14	6.14
	FS	-	-
	CoT	2.63	1.75
CodeLlama (13B)	ZS	21.92	7.89
	FS	2.63	0.87
	CoT	3.50	3.50

ZS: Zero-Shot, FS: Few-Shot, CoT: Chain-of-Thought

larger LLMs. Table 5 shows that fine-tuning substantially improves syntactic accuracy for several smaller models, most notably Phi-4 and CodeLlama, while semantic accuracy improves more modestly.

Error taxonomy analysis, reported in Tables 3 and 6, reveals that navigation errors and logical inconsistencies remain prevalent across both large and fine-tuned models. While fine-tuning reduces syntactic failures, higher-level semantic errors—such as incorrect constraint logic or invalid navigation paths—persist, indicating that improvements in surface correctness do not necessarily translate into deeper semantic understanding.

Table 5: Fine-tuned LLMs On Non-Transformed Models

Model	Syntactic Accuracy (%)	Semantic Accuracy (%)
Microsoft Phi-4 (14B)	75.43	48.24
Qwen-3 (8B)	38.59	22.80
Ministral-8B-Instruct (8B)	9.64	6.14
Llama-3 (8B)	64.91	36.84
CodeLlama (13B)	77.19	38.59

The RQ1 results serve as a diagnostic baseline, revealing systematic weaknesses in current LLM-based OCL generation even under favourable, non-transformed conditions. While large models and fine-tuned smaller models achieve high syntactic validity, error analysis shows that navigation errors, incorrect logical formulations, and invalid operation usage remain prevalent. Crucially, these failures often occur despite the preservation of original UML identifiers and structural patterns, suggesting that many models rely on syntactic familiarity rather than a robust understanding of the underlying model semantics.

These observations motivate the need for a stronger evaluation setting that isolates whether LLMs genuinely reason over UML structure and specification intent, or instead exploit memorized identifiers and recurring modelling patterns. To this end, we introduce semantics-preserving UML transformations that deliberately disrupt lexical and structural cues while maintaining the original specification meaning. RQ2 leverages this transformation-based setup to directly probe the generalization limitations exposed in RQ1.

Table 6: Error Taxonomy Distribution Across Fine-Tuned LLMs and Prompting Techniques

	Phi-4	Qwen-3	Mini-8B	Llama-3	CodeLlama
Navigation	14	10	2	19	9
Incorrect Ops	8	2	3	10	9
Hallucination	13	-	-	6	10
Logic	18	18	4	26	34
Invalid Ops	5	5	44	9	6
Not Generated	-	53	55	-	-

Phi-4 = Microsoft Phi-4 (14B), Qwen-3 = Qwen-3 (8B), Mini-8B = Ministral-8B-Instruct (8B).

5.2 RQ2: How do semantics-preserving UML model transformations affect the ability of LLMs to generate correct OCL expressions?

Building on RQ1, RQ2 investigates whether the syntactic and semantic weaknesses persist when syntactic cues are systematically removed. While RQ1 showed that LLMs often generate syntactically valid yet semantically incorrect OCL on original UML models, it remained unclear whether these errors reflect genuine misunderstanding or reliance on familiar identifiers and model structure. RQ2 addresses this by evaluating models on semantics-preserving UML transformations (Section 3), ensuring that any performance degradation stems from sensitivity to structural variation rather than changes in specification intent. Unlike RQ1, RQ2 focuses exclusively on large, general-purpose LLMs, positioning it as a stress test of generalization rather than a broad model comparison; given that smaller fine-tuned models already underperform on original inputs, their inclusion would add limited insight. This design sets an upper bound on robustness, allowing observed failures to be attributed to fundamental limitations rather than model capacity or fine-tuning.

Table 7 shows a substantial degradation in both syntactic and semantic accuracy across all evaluated models under transformation. For closed-source models, the best semantic accuracy decreases from 74.56% on non-transformed models to 55.96% on transformed models. Open-source models exhibit an even steeper decline, with semantic accuracy dropping to a maximum of 42.20%. These results demonstrate that even when transformations preserve the original specification intent, model performance deteriorates.

Across prompting strategies, no consistent robustness gains are observed. Few-shot prompting improves performance for certain models (e.g., GPT-5 and Llama-4) but severely degrades performance for others, such as Grok-4 Fast. Chain-of-thought prompting, which occasionally benefited syntactic accuracy in RQ1, often leads to lower semantic accuracy under transformation, indicating that additional reasoning steps do not compensate for disrupted lexical cues.

A closer examination of the error taxonomy in Table 8 reveals that invalid constraints constitute a dominant failure mode for several models. These invalid cases occur when the generated OCL references classes, attributes, associations, or role names that do not exist in the transformed UML model. Importantly, this behaviour persists despite the transformed UML model being explicitly provided as part of the prompt.

Manual inspection of these invalid constraints shows that many models continue to generate OCL expressions using identifiers from

Table 7: Results On Transformed Models

Model	Prompt	Syntactic Accuracy (%)	Semantic Accuracy (%)
Closed-Source Models			
Claude Sonnet 4.5	ZS	64.22	55.96
	FS	55.96	48.62
	CoT	63.30	52.29
GPT-5	ZS	67.88	55.96
	FS	66.05	57.79
	CoT	58.71	42.20
Gemini 2.5 Pro	ZS	69.72	56.88
	FS	63.30	55.04
	CoT	69.72	55.96
Grok-4 Fast	ZS	63.30	55.96
	FS	24.77	19.26
	CoT	46.78	40.36
Open-Source Models			
Llama-4	ZS	49.54	32.11
	FS	54.12	35.77
	CoT	55.96	34.86
DeepSeek V3.1	ZS	25.68	21.10
	FS	42.20	36.69
	CoT	54.12	42.20

ZS: Zero-Shot, FS: Few-Shot, CoT: Chain-of-Thought

the original, non-transformed models. Under the chain-of-thought prompting setting, 27 out of 114 invalid constraints produced were unchanged OCL expression, while under the few-shot prompting setting this behaviour was observed in 53 out of 114 cases. These findings indicate that a substantial portion of invalid outputs arise from models failing to adapt identifier usage to the transformed model. This suggests that the models rely heavily on learned associations between natural language descriptions and familiar UML naming patterns, rather than grounding constraint generation in the concrete model instance supplied at inference time. As a result, even semantics-preserving renaming are sufficient to invalidate the generated constraints.

Navigation and logic errors also increase under transformation, further indicating that structural alterations—such as reification of attributes into classes or associations—disrupt the models’ ability to correctly traverse the class model. These errors frequently manifest as incorrect navigation paths or misplaced quantifiers, leading to constraints that are syntactically valid but semantically misaligned with the intended specification.

RQ2 shows that current LLMs generalize poorly when UML models deviate from familiar lexical or structural patterns. The high rate of invalid constraints indicates that OCL generation often relies on syntactic cues rather than a model-grounded understanding of UML semantics, reinforcing RQ1 findings. When lexical and structural familiarity is disrupted, models struggle to maintain syntactic and semantic correctness, despite unchanged specification intent. These results highlight the need for transformation-based benchmarks and caution against overestimating performance on unmodified, publicly available models.

5.3 RQ3: Which types of OCL generation errors are revealed or amplified under transformation-based benchmarks?

RQ3 investigates the nature of errors produced by LLMs during OCL generation, with the goal of understanding why constraints fail rather than merely whether they are correct. To this end, we

Table 8: Error Taxonomy Distribution Across Large LLMs and Prompting Techniques on Transformed UML

	Models and Prompting Techniques																							
	Claude 4.5						GPT-5						Llama-4						Gemini 2.5					
	Z	F	C	T	S	O	Z	F	C	T	S	O	Z	F	C	T	S	O	Z	F	C	T	S	O
Navigation	21	30	27	18	22	16	26	28	32	19	16	23	14	40	24	22	12	13	21	30	27	18	22	16
Incorrect Ops	13	7	11	8	1	1	17	8	8	12	13	6	3	1	12	3	6	13	13	7	11	8	1	1
Hallucination	1	1	4	4	2	2	5	12	12	5	1	9	2	4	8	5	3	5	1	1	4	4	2	2
Logic	8	7	8	9	7	16	14	8	11	9	8	6	3	2	5	3	3	2	8	7	8	9	7	16
Invalid Ops	6	11	2	8	11	29	12	14	8	3	4	5	62	16	7	15	61	32	6	11	2	8	11	29

ZS: Zero-Shot, FS: Few-Shot, CoT: Chain-of-Thought

classify incorrect constraints using a taxonomy comprising navigation errors, incorrect operations, hallucinations, logical errors, and invalid operations.

Table 8 summarizes the distribution of error types across models and prompting strategies on transformed UML models. Analysis of the error taxonomy reveals that LLM failures are systematic and closely tied to reasoning mechanisms. Navigation errors, which dominate across most models, are not merely a consequence of altered UML structure but indicate a deeper reliance on memorized paths and identifiers rather than adaptive traversal logic. These errors often reflect the model’s inability to generalize constraint navigation when class associations or roles are recontextualized, highlighting a reliance on lexical cues over structural understanding.

A second prominent failure category is invalid operations, which dominate the error distribution for several models. Invalid operations frequently correspond to references to non-existent classes, attributes, associations, or role names. As discussed in RQ2, these errors often stem from the model reverting to identifiers from the original UML model rather than grounding generation in the transformed model provided in the prompt. This behaviour is especially pronounced for certain prompting strategies, where invalid operation counts increase sharply despite otherwise reasonable syntactic structure.

Logical errors and incorrect operation usage appear consistently across models but at lower frequencies. These errors include misplaced quantifiers, incorrect collection operations, or improper use of OCL operators such as exists, forAll, or size. While these constraints may be syntactically valid, they fail to capture the intended semantics described in the natural language specification. The persistence of such errors indicates that reasoning about constraint semantics remains challenging even when high-level syntax is correct.

Hallucination errors, though less frequent overall, are more prevalent in some open-source models and under chain-of-thought prompting. These errors involve introducing constraints, attributes, or relationships not mentioned in either the UML model or the specification. This suggests that encouraging intermediate reasoning steps can sometimes amplify speculative generation rather than improve grounding.

RQ3 reveals that OCL generation errors are not random but exhibit systematic patterns tied to lexical reliance, structural sensitivity, and incomplete semantic grounding. The prevalence of navigation and invalid operation errors reinforces the conclusion that current LLMs often rely on syntactic pattern matching instead

of constructing constraints grounded in the concrete UML model instance.

5.4 RQ4: To what extent do individual and combined UML transformations differentially affect LLM performance in OCL generation?

To investigate the individual and combined impact of semantics-preserving transformations on LLM-generated OCL expressions, we conducted an ablation study using GPT-5 and Llama-4 under a chain-of-thought prompting strategy. Table 9 reports the syntactic and semantic accuracy for single and combined transformations, while Table 10 presents the distribution of error types. These results reveal nuanced insights into how different types of transformations affect model reasoning and the association between lexical, attribute-level, and structural modifications.

For single transformations, syntactic accuracy remains relatively high across both models, with association reification achieving the highest performance (78.94% for GPT-5), suggesting that models can generally produce well-formed OCL expressions even when structural elements are altered. In contrast, semantic accuracy is consistently lower and more sensitive to transformations, indicating that understanding and correctly reasoning about the model’s structure is more brittle than generating syntactically correct expressions. Among the transformations, renaming and attribute reification cause more pronounced drops in semantic accuracy (64.91% and 62.28% for GPT-5, respectively), highlighting that lexical and attribute-level changes disrupt the model’s ability to resolve references and interpret attribute semantics more than purely structural modifications.

When transformations are combined, performance degradation becomes more pronounced, particularly for semantic accuracy. This indicates that the cumulative effects of multiple transformations compound reasoning difficulty, rather than any single transformation being solely responsible. Notably, the advantage of association reification observed in isolation diminishes in combined settings, suggesting that structural modifications are manageable when isolated but interact with lexical and attribute changes in ways that amplify model errors.

Table 9: Impact of Individual and Combined Transformations on Syntactic and Semantic Accuracy (%)

Transformation	GPT-5		Llama-4	
	Syntax	Semantics	Syntax	Semantics
Single Transformations				
Renaming	73.68 (+14.97%)	64.91 (+22.71%)	62.63 (+6.67%)	40.35 (+5.49%)
Attribute	69.29 (+10.58%)	62.28 (+20.08%)	63.50 (+7.54%)	41.98 (+7.12%)
Association	78.94 (+30.23%)	68.42 (+26.22%)	60.52 (+4.56%)	42.10 (+7.24%)
Combined Transformations				
Renaming + Attribute	68.42 (+9.71%)	60.52 (+18.32%)	67.36 (+11.40%)	38.59 (+3.73%)
Renaming + Association	68.42 (+9.71%)	60.52 (+18.32%)	64.73 (+8.77%)	34.95 (+0.09%)
Attribute + Association	73.68 (+14.97%)	64.91 (+22.71%)	62.10 (+6.14%)	35.96 (+1.10%)
Renaming + Attribute + Association	58.71	42.20	55.96	34.86

The error distribution in Table 10 provides further insight into the underlying mechanisms of model failure. Navigation errors dominate across nearly all configurations, particularly when renaming

is applied, reflecting the models’ difficulty in resolving references after lexical alterations. Attribute reification is associated with a higher frequency of invalid and hallucinated expressions, showing that models struggle to correctly interpret reified attributes and may introduce elements not present in the transformed model. In contrast, association reification produces fewer navigation errors in isolation, but when combined with other transformations, it contributes to a rise in invalid and operational errors, suggesting that models can handle structural adjustments individually but fail to integrate them with concurrent lexical and attribute-level changes. GPT-5 and Llama-4 exhibit similar trends, though Llama-4 appears more sensitive to semantic disruption, particularly in combined transformations, highlighting differences in model robustness to complex reasoning tasks.

Overall, the ablation study demonstrates that performance degradation in the full transformation scenario arises from interactions between multiple dimensions of reasoning rather than the effect of a single transformation. Lexical and attribute-level transformations primarily challenge the models’ ability to accurately interpret and navigate the UML elements, while structural transformations mainly affect operational and logic reasoning when compounded.

Table 10: Error Taxonomy Distribution per Transformation

	GPT-5					Llama-4				
	Hal	Nav	Log	Inc	Inv	Hal	Nav	Log	Inc	Inv
Single Transformations										
Renaming	5	28	5	3	-	6	24	8	7	25
Attribute	2	12	6	5	14	7	29	5	1	24
Association	6	7	6	5	13	10	29	1	8	18
Combined Transformations										
Renaming + Attribute	6	22	3	1	11	5	30	5	8	22
Renaming + Association	7	21	2	3	13	10	36	2	7	21
Attribute + Association	5	11	5	3	14	7	38	-	4	25

Hal: Hallucination, Nav: Navigation, Log: Logic, Inc: Incorrect Ops, Inv: Invalid Ops

6 Discussion

Our study reveals several key insights into the robustness of LLMs for OCL generation under semantics-preserving UML transformations. Across models and prompting strategies, performance consistently declines when identifiers are renamed or structural elements are reified, suggesting that models rely substantially on lexical and surface-level cues.

The ablation study highlights distinct patterns of failure. Identifier renaming predominantly triggers navigation and reference errors, indicating reliance on familiar naming conventions rather than on model comprehension of UML relationships. Attribute and association reification introduce additional structural complexity, which disproportionately affects the construction of multi-object constraints. Combined transformations reveal that errors are not isolated: failures accumulate in a non-linear fashion, suggesting that models struggle with compositional reasoning over altered structures.

A closer examination of error types uncovers three dominant failure modes. First, models frequently navigate incorrect associations after reification, producing constraints that are syntactically valid

but semantically inconsistent with the underlying model. Second, residual lexical dependencies remain evident, as some models attempt to leverage original naming semantics even when identifiers are obfuscated. Third, incomplete constraint generation is common under structural perturbations, particularly for constraints involving multi-class interactions or nested conditions. Together, these patterns indicate that model performance is highly sensitive to the structural cues embedded in the original UML representation.

Beyond the high-level error categories, a finer-grained qualitative analysis reveals sub-patterns within several error classes. Invalid operations frequently stem from missing or incorrect use of higher-order OCL operators like `isUnique()` or from incomplete expressions lacking required iterator variables or return types, leading to syntactic or semantic invalidity. Navigation errors often arise when class names are replaced with association names; while role-based navigation is valid in OCL, generated identifiers are often incorrect or inconsistent leading to broken navigation paths, reflecting reliance on inferred naming patterns rather than grounded model structure. Furthermore, hallucinations are particularly common in constraints involving temporal concepts, where non-existent OCL operations or unsupported constructs are introduced.

These findings also suggest several directions for improving LLM-based OCL generation. First, future models should incorporate explicit representations of UML structure, such as graph-based encoders or retrieval mechanisms that ground generation in the supplied class model rather than relying primarily on lexical identifiers. Second, transformation-augmented fine-tuning using semantics-preserving UML refactorings could improve robustness to alternative model representations while reducing dependence on recurring naming patterns. Finally, validity-aware generation strategies, such as iterative verification against UML constraints or automatic repair using tools like USE, could reduce navigation and typing errors before returning the final OCL expression. More broadly, transformation-based benchmarks provide a diagnostic framework for measuring model grounding and can guide the development of more robust LLMs for model-driven engineering.

7 Threats to Validity

Construct Validity. This work evaluates measurement validity using syntactic and semantic accuracy. Syntactic accuracy measures conformity to OCL grammar, while semantic accuracy assesses alignment with the intended specification. Semantics are evaluated by executing constraints on manually constructed object instances that capture satisfying and violating cases. Although these instances are derived from and validated against the natural language specification, they are limited in scope. Gold OCL constraints are also used as a consistency check, potentially introducing bias. Additionally, transformations are based on observed LLM error patterns, so performance reflects robustness to known weaknesses rather than general transformation resilience.

Internal Validity. A key concern is whether results are influenced by confounding factors. Prompt design may affect outcomes, so we use consistent templates for zero-shot, few-shot, and chain-of-thought strategies. Transformation correctness is another risk, where although designed to preserve semantics, errors are possible.

We mitigate this by automatically rewriting OCL constraints, checking transformed models on test instances, and manually inspecting samples, though this validation is empirical and limited by instance coverage. Finally, fine-tuning is only applied to smaller open-source models, but large models—already performing best—also show degradation, supporting our conclusions.

External Validity. Our results are limited by the dataset and transformations used. The evaluation covers a finite set of UML models and specifications, which may not reflect all real-world practices. While we test both open and closed-source LLMs of different sizes, findings may not apply to future or UML-specialized models. Transformations focus on common refactorings like renaming and reification, so other changes could expose additional failures. Thus, observed performance drops indicate general robustness issues, not a complete picture.

8 Related works

Automatically generating OCL constraints from natural-language requirements is a key challenge in model-driven engineering, as these constraints are essential for ensuring model correctness, enabling formal verification, and supporting downstream analysis. Early approaches relied on rule-based systems, controlled natural languages, and template-driven techniques [7]. While effective in constrained settings, these methods require significant manual effort and do not generalize across diverse UML structures and specification styles. Traditional machine learning approaches have also been explored, but their reliance on handcrafted features and limited training data restricts their scalability and applicability in realistic modeling scenarios [5, 42].

Large language models (LLMs) introduce new capabilities for automating OCL generation by enabling end-to-end mapping from natural language to formal specifications [12, 27, 39]. Prior work shows that LLMs can produce syntactically well-formed OCL expressions when guided by UML models and carefully designed prompts [3, 36], with further improvements achieved through prompt engineering and domain-specific fine-tuning [33, 43]. However, syntactic correctness does not guarantee semantic validity: generated constraints frequently exhibit navigation errors, misuse of OCL operations, hallucinated model elements, or fail to capture the intended specification semantics [35].

Evaluating LLMs on this task presents additional challenges. Existing benchmarks are scarce and often limited to small, static datasets pairing UML models, natural-language requirements, and gold-standard OCL constraints [4, 8, 32]. This increases the risk of memorization, data leakage, and overfitting [11, 20], where correct outputs may reflect familiarity with recurring modeling patterns rather than true semantic reasoning [44]. As a result, current evaluation practices may overestimate LLM capabilities.

To address similar limitations, prior work in software testing, program analysis, and code generation advocates for robustness evaluation through semantics-preserving transformations [16, 18, 21]. Simple transformations, such as identifier renaming or structural refactoring, often expose significant brittleness in LLMs that standard benchmarks fail to reveal [23, 24]. In model-driven engineering, UML transformations have long been used for refactoring and model evolution while preserving semantics [30], making them

a natural foundation for constructing more diagnostic evaluation scenarios.

Complementary work emphasizes structured error analysis to better understand LLM limitations [2, 41]. Prior studies distinguish between syntactic, semantic, and logical errors in formal artifact generation [40]. Building on this, our exploratory analysis identifies recurring OCL-specific errors, including navigation mistakes, misuse of OCL operations, hallucinated elements, semantic mismatches, and invalid constructs. These patterns persist across model sizes, prompting strategies, and fine-tuning regimes, indicating systematic limitations rather than isolated failures.

Despite these insights, existing benchmarks rarely expose such weaknesses in a principled manner [13], often allowing models to succeed without requiring deep, model-aware reasoning [34]. In this paper, we leverage empirically grounded error patterns to guide the construction of transformed UML benchmarks. By applying deterministic, semantics-preserving transformations—such as identifier renaming, attribute reification, and association reification [6, 28, 38]—that target known weaknesses, we enable more robust evaluation, mitigate data leakage effects, and provide a clearer assessment of LLM capabilities in generating OCL constraints.

9 Conclusion

This paper examined the robustness of LLMs in generating OCL specifications from natural language requirements and UML class models, showing that standard benchmarks can overestimate model capability by overlooking systematic semantic and structural weaknesses. Our baseline analysis found that both closed-source and open-source models achieve high syntactic correctness on original UML models, yet semantic errors—navigation mistakes, logical mismatches, and incorrect OCL operations—remain common and are only partially mitigated by fine-tuning or advanced prompting. To address this, we introduced a transformation-driven benchmark using deterministic, semantics-preserving UML transformations that remove syntactical cues while preserving specification intent. These transformations caused substantial drops in both syntactic and semantic accuracy, exposing LLM brittleness. Error analysis showed that transformed benchmarks amplify recurring failure modes, particularly navigation errors and logical inconsistencies, confirming that the approach reveals latent weaknesses rather than increasing task difficulty. Overall, UML model transformations serve not as data augmentation but as a principled, fault-revealing benchmark design that more reliably assesses LLM generalization and reliance on memorization. Future work includes exploring transformation-aware training, validity-aware learning objectives, and exposure to transformation-augmented datasets to improve semantic rigour, robustness to identifier and structural changes, and predictive accuracy of real-world model deployment.

References

- [1] 2014. On the verification of UML/OCL class diagrams using constraint programming. *Journal of Systems and Software* 93 (2014), 1–23.
- [2] Altaf Allah Abbassi, Leuson Da Silva, Amin Nikanjam, and Foutse Khomh. 2025. Unveiling inefficiencies in llm-generated code: Toward a comprehensive taxonomy. *arXiv preprint arXiv:2503.06327* (2025).
- [3] Seif Abukhalaf, Mohammad Hamdaqa, and Foutse Khomh. 2023. On Codex prompt engineering for OCL generation: an empirical study. In *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*. IEEE, 148–157.
- [4] Seif Abukhalaf, Mohammad Hamdaqa, and Foutse Khomh. 2024. PathOCL: path-based prompt augmentation for OCL generation with GPT-4. In *Proceedings of the 2024 IEEE/ACM First International Conference on AI Foundation Models and Software Engineering*. 108–118.
- [5] Shaikat Ali, Muhammad Zohaib Iqbal, Andrea Arcuri, and Lionel C Briand. 2013. Generating test data from OCL constraints with search techniques. *IEEE Transactions on Software Engineering* 39, 10 (2013), 1376–1402.
- [6] Thomas Baar and Slaviša Marković. 2006. A graphical approach to prove the semantic preservation of UML/OCL refactoring rules. In *International Andrei Ershov Memorial Conference on Perspectives of System Informatics*. Springer, 70–83.
- [7] Imran Sarwar Bajwa, Behzad Bordbar, and Mark G Lee. 2010. OCL constraints generation from natural language specification. In *2010 14th IEEE International Enterprise Distributed Object Computing Conference*. IEEE, 204–213.
- [8] Jordi Cabot, David Delgado, and Lola Burgueño. 2022. Combining OCL and natural language: a call for a community effort. In *Proceedings of the 25th International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings*. 908–912.
- [9] Jordi Cabot and Martin Gogolla. 2012. Object constraint language (OCL): a definitive guide. In *International school on formal methods for the design of computer, communication and software systems*. Springer, 58–90.
- [10] Wei Wei Cai, Nian Yang, and Bo Li. 2026. Performance of multimodal large language model in understanding digital UML class diagram. *Enterprise Information Systems* (2026), 2629556.
- [11] Wentao Chen, Lizhe Zhang, Li Zhong, Letian Peng, Zilong Wang, and Jingbo Shang. 2025. Memorize or generalize? evaluating llm code generation with evolved questions. *arXiv preprint arXiv:2503.02296* (2025).
- [12] Yoonsik Cheon. 2025. LLMs as Code Generators for Model-Driven Development. In *Proceedings of the 20th International Conference on Software Technologies (ICSOFT)*. 386–393.
- [13] Nurit Cohen-Inger, Yehonatan Elisha, Bracha Shapira, Lior Rokach, and Seffi Cohen. 2025. Forget What You Know about LLMs Evaluations—LLMs are Like a Chameleon. *arXiv preprint arXiv:2502.07445* (2025).
- [14] David Delgado, Lola Burgueño, and Robert Clarisó. 2026. A framework for assessing the capabilities of code generation of constraint domain-specific languages with large language models. *arXiv preprint arXiv:2603.05278* (2026).
- [15] Nouha Dziri, Ximing Lu, Melanie Sclar, Xiang Lorraine Li, Liwei Jiang, Bill Yuchen Lin, Sean Welleck, Peter West, Chandra Bhagavatula, Ronan Le Bras, et al. 2023. Faith and fate: Limits of transformers on compositionality. *Advances in neural information processing systems* 36 (2023), 70293–70332.
- [16] Timothy Elvira, Tyler Thomas Procko, Lynn Vonderhaar, and Omar Ochoa. 2024. Exploring Testing Methods for Large Language Models. In *2024 International Conference on Machine Learning and Applications (ICMLA)*. IEEE, 1152–1157.
- [17] Sarah Fakhoury, Aaditya Naik, Georgios Sakkas, Saikat Chakraborty, and Shuvendu K Lahiri. 2024. LLM-based test-driven interactive code generation: User study and empirical evaluation. *IEEE Transactions on Software Engineering* (2024).
- [18] Shiwei Feng, Xiangzhe Xu, Xuan Chen, Kaiyuan Zhang, Syed Yusuf Ahmed, Zian Su, Mingwei Zheng, and Xiangyu Zhang. 2025. IntenTest: Stress Testing for Intent Integrity in API-Calling LLM Agents. *arXiv preprint arXiv:2506.07524* (2025).
- [19] Martin Gogolla, Fabian Büttner, and Mark Richters. 2007. USE: A UML-based specification environment for validating UML and OCL. *Science of Computer Programming* 69, 1-3 (2007), 27–34.
- [20] Katie Kang, Amrith Setlur, Dibya Ghosh, Jacob Steinhardt, Claire Tomlin, Sergey Levine, and Aviral Kumar. 2024. What Do Learning Dynamics Reveal About Generalization in LLM Reasoning? *arXiv preprint arXiv:2411.07681* (2024).
- [21] Man Ho Lam, Chaozheng Wang, Jen-tse Huang, and Michael R Lyu. 2025. CODE-CRASH: Stress Testing LLM Reasoning under Structural and Semantic Perturbations. *arXiv preprint arXiv:2504.14119* (2025).
- [22] Jiatong Li, Rui Li, and Qi Liu. 2023. Beyond static datasets: A deep interaction approach to llm evaluation. *arXiv preprint arXiv:2309.04369* (2023).
- [23] Yichen Li, Yintong Huo, Zhihan Jiang, Renyi Zhong, Pinjia He, Yuxin Su, Lionel C Briand, and Michael R Lyu. 2024. Exploring the effectiveness of llms in automated logging statement generation: An empirical study. *IEEE Transactions on Software Engineering* (2024).
- [24] Ziyu Li and Donghwan Shin. 2024. Mutation-based consistency testing for evaluating the code understanding capability of llms. In *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering-Software Engineering for AI*. 150–159.
- [25] José Antonio Hernández López, Boqi Chen, Moteez Saad, Tushar Sharma, and Dániel Varró. 2024. On inter-dataset code duplication and data leakage in large language models. *IEEE Transactions on Software Engineering* (2024).
- [26] Riccardo Lunardi, Vincenzo Della Mea, Stefano Mizzaro, and Kevin Roitero. 2025. On robustness and reliability of benchmark-based evaluation of llms. *arXiv preprint arXiv:2509.04013* (2025).
- [27] Yuyu Luo, Guoliang Li, Ju Fan, Chengliang Chai, and Nan Tang. 2025. Natural language to sql: State of the art and open problems. *Proceedings of the VLDB Endowment* 18, 12 (2025), 5466–5471.

- [28] Tiago Massoni, Rohit Gheyi, and Paulo Borba. 2005. Formal refactoring for UML class diagrams. In *Simpósio Brasileiro de Engenharia de Software (SBES)*. SBC, 152–167.
- [29] Christoph Mayr-Dorn, Anmol Bilal, Cosmina Ratiu, and Alexander Egyed. 2025. Generating Quality Assurance Constraints From Natural Language With LLMs. *Journal of Software: Evolution and Process* 37, 11 (2025), e70062.
- [30] Mohammed Misbhaudhin and Mohammad Alshayeb. 2015. UML model refactoring: a systematic literature review. *Empirical Software Engineering* 20, 1 (2015), 206–251.
- [31] Object Management Group. 2014. *Object Constraint Language (OCL), Version 2.4*. Technical Report. OMG. <https://www.omg.org/spec/OCL/2.4/>
- [32] Fengjunjie Pan, Vahid Zolfaghari, Long Wen, Nenad Petrovic, Jianjie Lin, and Alois Knoll. 2024. Generative ai for ocl constraint generation: Dataset collection and llm fine-tuning. In *2024 IEEE International Symposium on Systems Engineering (ISSE)*. IEEE, 1–8.
- [33] Venkatesh Balavadhani Parthasarathy, Ahtsham Zafar, Aafaq Khan, and Arsalan Shahid. 2024. The ultimate guide to fine-tuning llms from basics to breakthroughs: An exhaustive review of technologies, research, best practices, applied research challenges and opportunities. *arXiv preprint arXiv:2408.13296* (2024).
- [34] Eva Sánchez Salido, Julio Gonzalo, and Guillermo Marco. 2025. None of the others: a general technique to distinguish reasoning from memorization in multiple-choice llm evaluation benchmarks. *arXiv preprint arXiv:2502.12896* (2025).
- [35] Eddie Antonio Santos, Joshua Charles Campbell, Dhvani Patel, Abram Hindle, and José Nelson Amaral. 2018. Syntax and sensibility: Using language models to detect and correct syntax errors. In *2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 311–322.
- [36] Hanan A Siala and Kevin Lano. 2025. Towards Using LLMs in the Reverse Engineering of Software Systems to Object Constraint Language. In *2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 1–6.
- [37] Mathias Soeken, Robert Wille, Mirco Kuhlmann, Martin Gogolla, and Rolf Drechsler. 2010. Verifying UML/OCL models using Boolean satisfiability. In *2010 Design, Automation & Test in Europe Conference & Exhibition (DATE 2010)*. IEEE, 1341–1344.
- [38] Guido Wachsmuth. 2007. Metamodel adaptation and model co-adaptation. In *European conference on object-oriented programming*. Springer, 600–624.
- [39] Jianxun Wang and Yixiang Chen. 2023. A review on code generation with llms: Application and evaluation. In *2023 IEEE International Conference on Medical Artificial Intelligence (MedAI)*. IEEE, 284–289.
- [40] Zhijie Wang, Zijie Zhou, Da Song, Yuheng Huang, Shengmai Chen, Lei Ma, and Tianyi Zhang. 2025. Towards understanding the characteristics of code generation errors made by large language models. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 717–717.
- [41] Liyan Xu, Zhenlin Su, Mo Yu, Jin Xu, Jinho D Choi, Jie Zhou, and Fei Liu. 2024. Identifying Factual Inconsistencies in Summaries: Grounding LLM Inference via Task Taxonomy. *arXiv preprint arXiv:2402.12821* (2024).
- [42] Yilong Yang, Yibo Liu, Tianshu Bao, Weiru Wang, Nan Niu, and Yongfeng Yin. 2024. DeepOCL: A deep neural network for Object Constraint Language generation from unrestricted nature language. *CAAI Transactions on Intelligence Technology* 9, 1 (2024), 250–263.
- [43] J Diego Zamfirescu-Pereira, Richmond Y Wong, Bjoern Hartmann, and Qian Yang. 2023. Why Johnny can't prompt: how non-AI experts try (and fail) to design LLM prompts. In *Proceedings of the 2023 CHI conference on human factors in computing systems*. 1–21.
- [44] Xin Zhou, Martin Weyssow, Ratnadira Widayarsi, Ting Zhang, Junda He, Yunbo Lyu, Jianming Chang, BeiQi Zhang, Dan Huang, and David Lo. 2025. Lessleak-bench: A first investigation of data leakage in llms across 83 software engineering benchmarks. *arXiv preprint arXiv:2502.06215* (2025).